Encodage et décodage d'une image à l'aide des codes correcteurs d'erreurs de Reed-Muller.

Prigent Devan N° 22017

1. Codes Correcteurs d'erreurs
   - Principe
   - Décodage d'un code
2. Code de Reed-Muller
   - Encodage
   - Décodage
3. Application
   - Protocole de l'expérience
   - Résultats

1. Codes Correcteurs d'erreurs
   - Principe
   - Décodage d'un code
2. Code de Reed-Muller
   - Encodage
   - Décodage
3. Application
   - Protocole de l'expérience
   - Résultats

Image du sol de Mars prise par la Sonde Mariner 9

Figure – Image de **Nirgal Vallis** prise par Mariner 9 [nasa.gov]

Problématique : Dans quelle mesure les codes de Reed-Muller permettent-ils de corriger ces erreurs en fonction de leur nombre et de leur localisation ?

Schéma de l'évolution d'un message au cours de la transmission

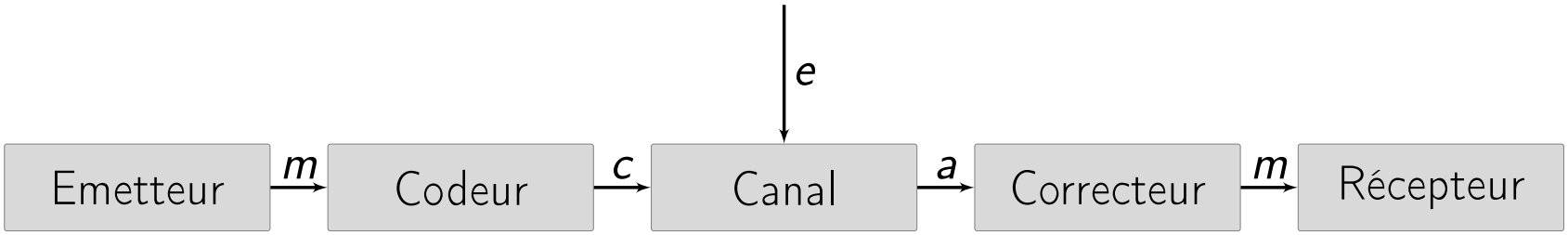
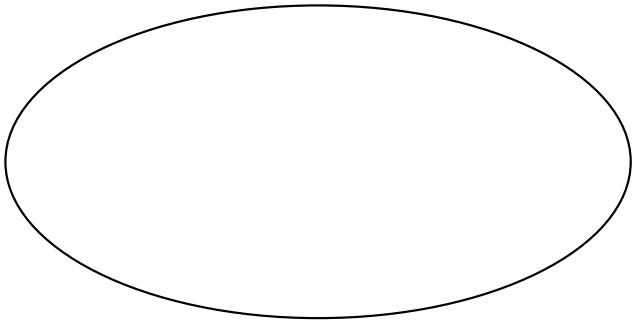


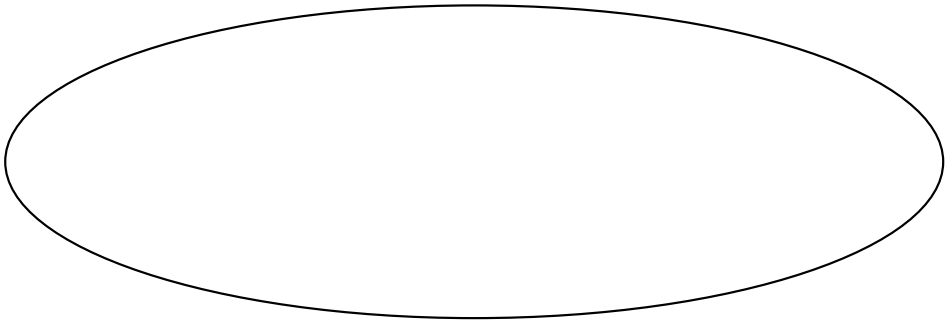
Figure – Représentation de la transmission du message

m : message initial
c : message codé
a : message altéré
e : erreur

Définitions

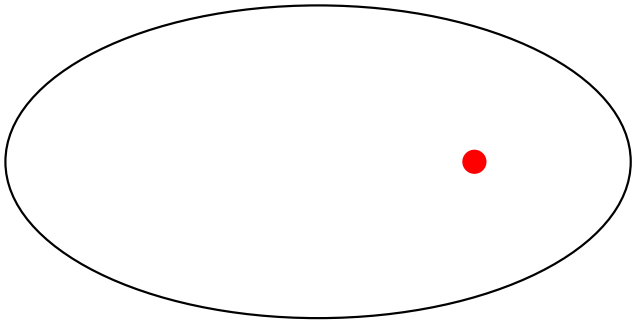


F_2^k

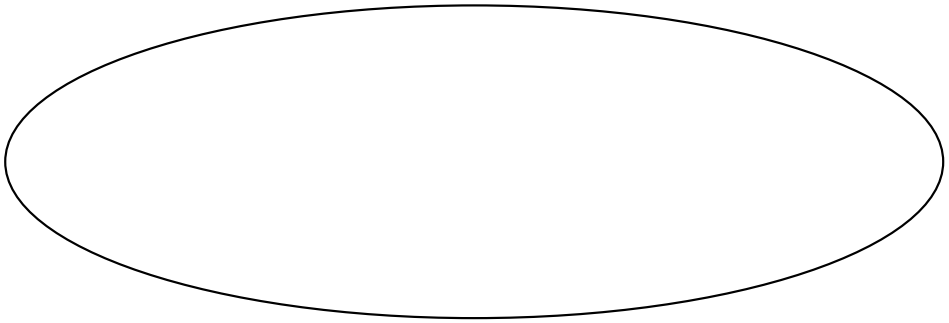


F_2^n

Définitions

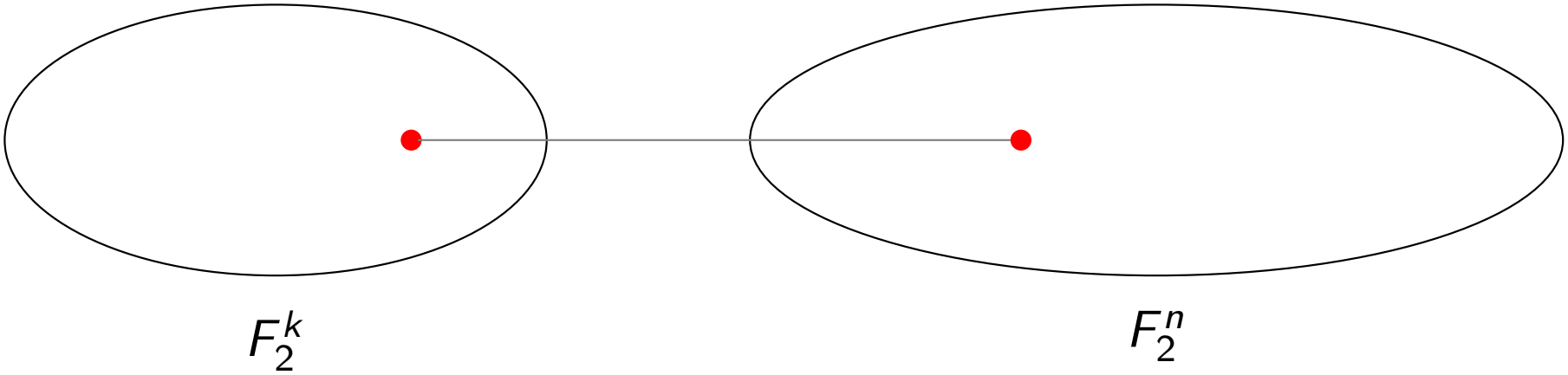


F_2^k

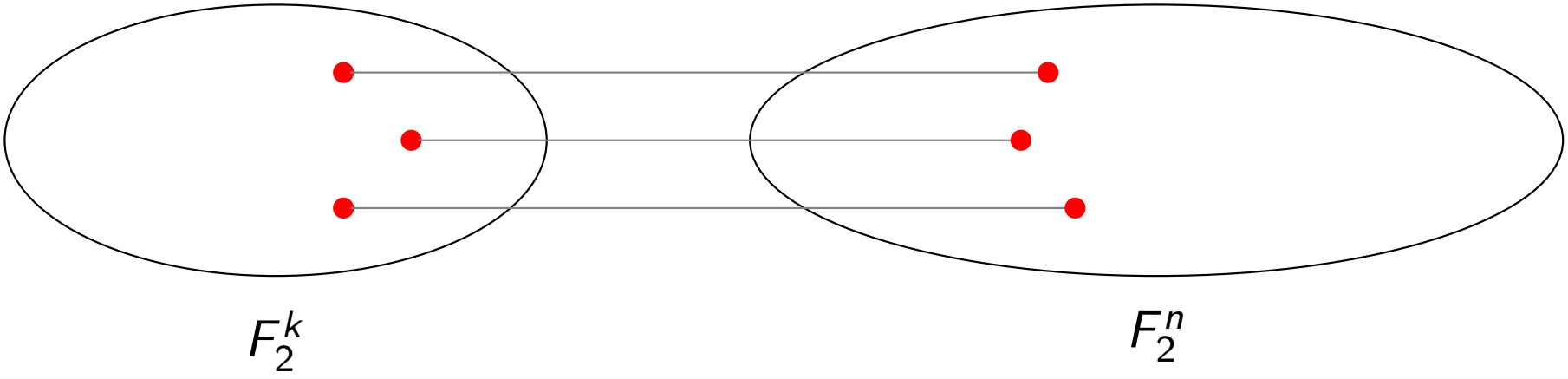


F_2^n

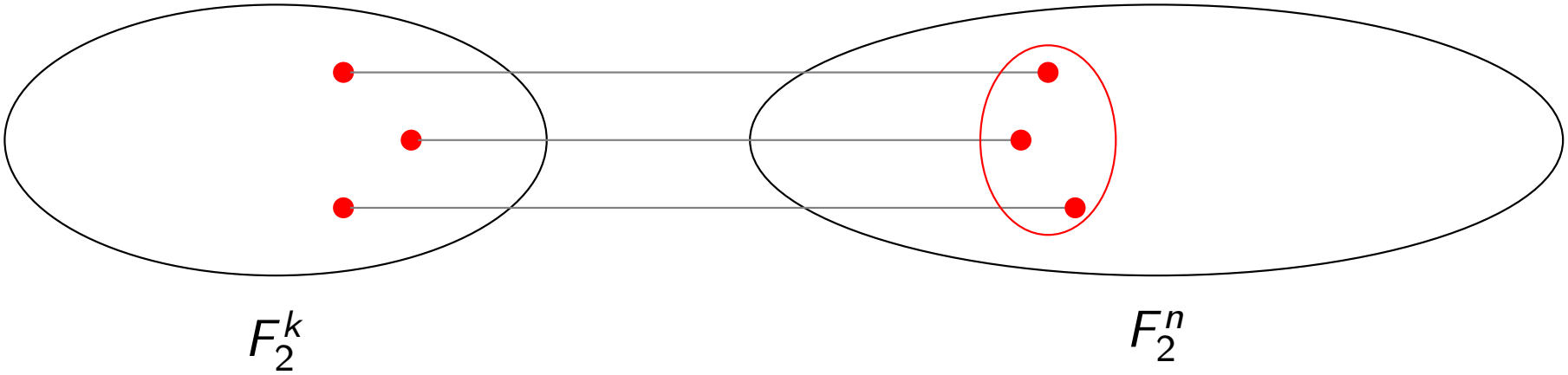
Définitions



Définitions



Définitions



Définitions

Code linéaire

Un code linéaire de longueur n et de dimension k est un sev C de dimension k de F_2^n.
On dit que C est le code de paramètres (n, k).

Poids

Soit m un mot de C, son poids $\omega(m)$ est le nombre de bits de m égaux à 1.

Décodage

Propriété

Une erreur est d'autant plus probable que son poids est faible.

message reçu : $a = m + e$

L'algorithme de décodage recherche le maximum de vraisemblance en minimisant $\omega(e)$, c'est-à-dire en choisissant $m \in C$ de manière à minimiser $\omega(a - m)$.

Distance de Hamming

Soit m et n deux mots de C, on définit $d(m,n) = \omega(m-n)$.

Distance minimale

La distance minimale d'un code C est définie par :
$d = \min \left\{ d(x,y) | (x,y) \in C^2, x \neq y \right\}$

Détection des erreurs

On peut alors détecter toute erreur telle que $\omega(e) < d$.
Si un mot m de C est perturbé par une erreur de poids non-nul, alors le mot reçu m' n'appartient pas à C car sinon

$$e = m' - m \in C$$

ce qui est absurde.

Schéma représentant des boules de Hamming dans F_2^n

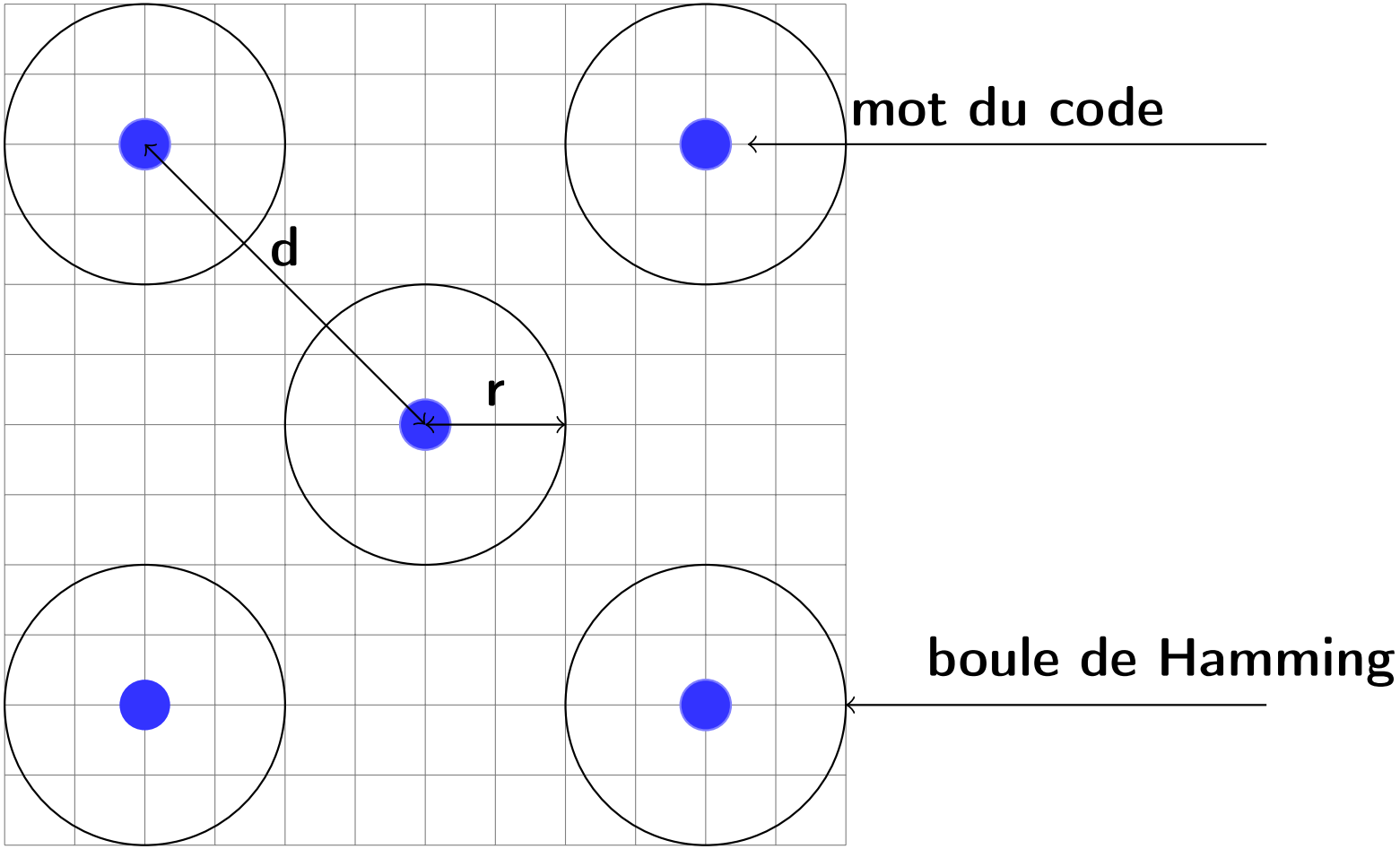
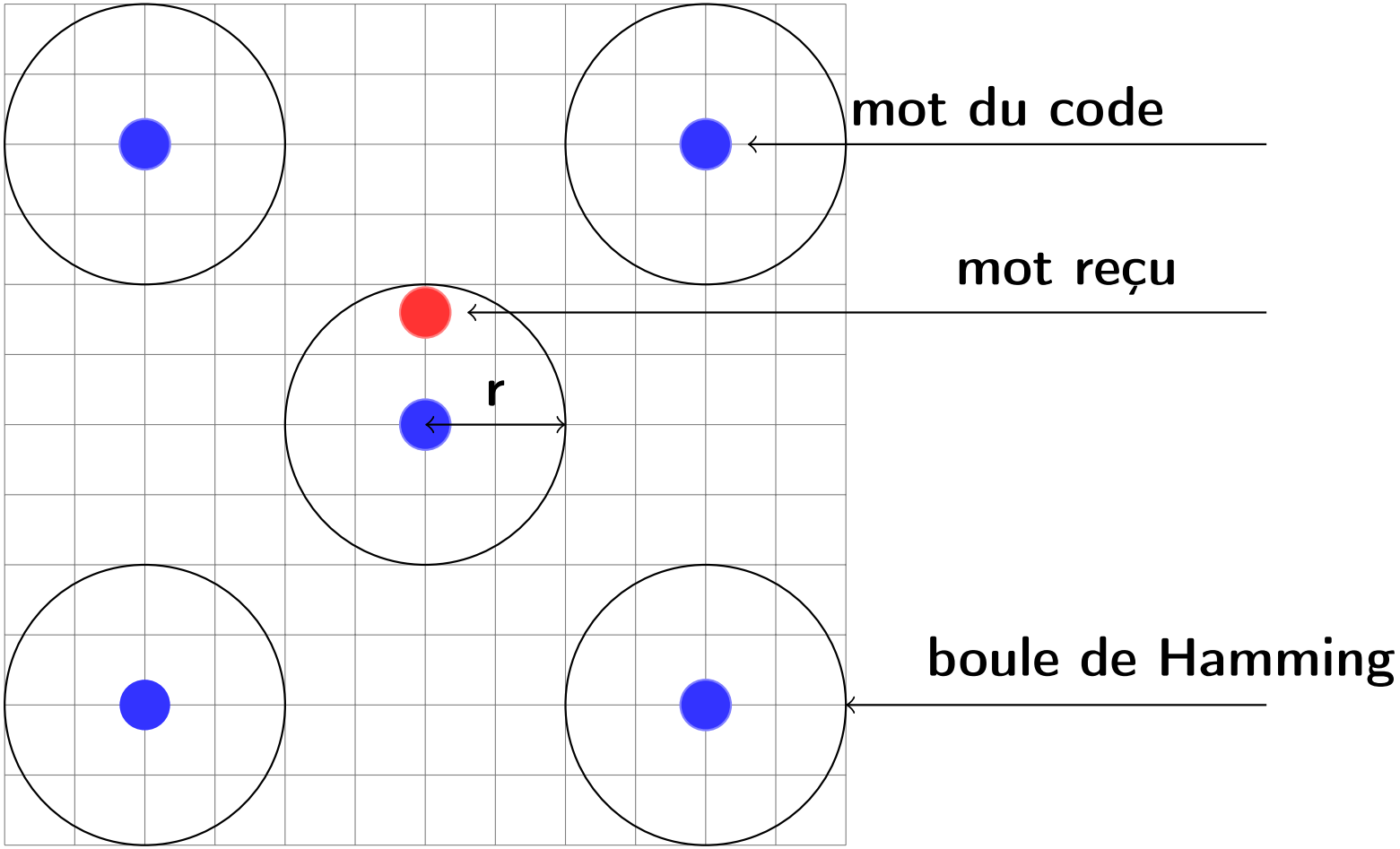


Schéma représentant des boules de Hamming dans F_2^n



1 Codes Correcteurs d'erreurs
- Principe
- Décodage d'un code

2 Code de Reed-Muller
- Encodage
- Décodage

3 Application
- Protocole de l'expérience
- Résultats

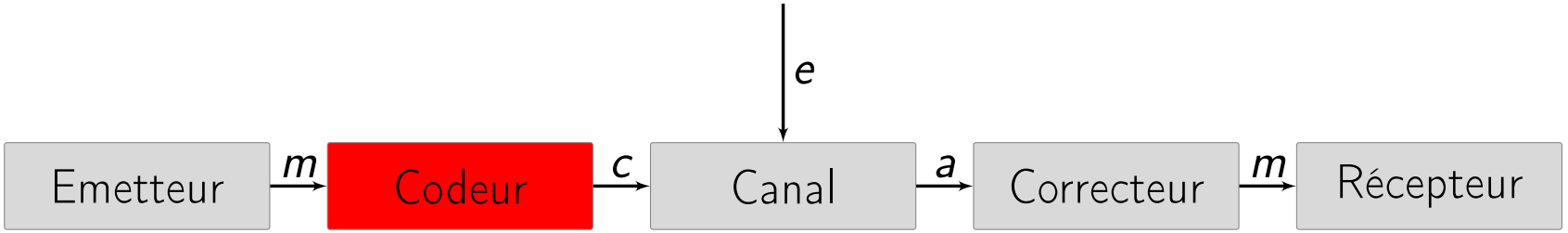


Figure – Représentation de la transmission du message

Définition d'un code RM

Les codes de Reed-Muller sont des codes linéaires décrits par 2 paramètres :

- r : ordre
- m : nombre de variable.

Codage

Matrice génératrice

Soit C un code de paramètres (n, k), il existe ϕ une application linéaire de F_2^k dans F_2^n telle que $Im(\phi) = C$.

Création de la matrice génératrice

Ensemble F_2^m

$F_2^m = \{z_0, z_1, z_2, ..., z_{2^m-1}\}$ où pour tout $i \in [0, 2^m - 1]$, z_i est la décomposition en binaire de l'entier i.

Hyperplans H_i

$H_i = \{(x_0, x_1, x_2, ..., x_{m-1}) \in F_2^m, x_i = 0\}$.

Fonction indicatrice $\mathbb{1}$

$$\mathbb{1} : H_i \mapsto \mathbb{1}_{H_i}$$

où

$$(\mathbb{1}_{H_i})_j = \begin{cases} 1 \text{ si } z_j \in H_i \\ 0 \text{ sinon} \end{cases}$$

.

Vecteurs v_i

On définit $v_0 = \begin{pmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{pmatrix}$ et v_i où $\forall i \in [1, m]$, $v_i = \mathbb{1}_{H_i}$.

Le code de Reed-Muller (r, m) de longueur 2^m est le code généré par v_0 et le produit extérieur d'au plus r des v_i où $v_i = \mathbb{1}_{H_i}$.

$r = 2$ et $m = 3$

$$F_2^m = ([0,0,0],[0,0,1],[0,1,0],[0,1,1],[1,0,0],[1,0,1],[1,1,0],[1,1,1])$$

$H_1 = \{z_0, z_1, z_2, z_3\}$;
$v_1 = \mathbb{1}_{H_1} = \begin{pmatrix} 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 \end{pmatrix}$.

Matrice génératrice du code Reed Muller $RM(2,3)$

$$G_{RM} = \begin{pmatrix} 1 \\ x_0 \\ x_1 \\ x_2 \\ x_0x_1 \\ x_0x_2 \\ x_1x_2 \end{pmatrix} = \begin{pmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \end{pmatrix}$$

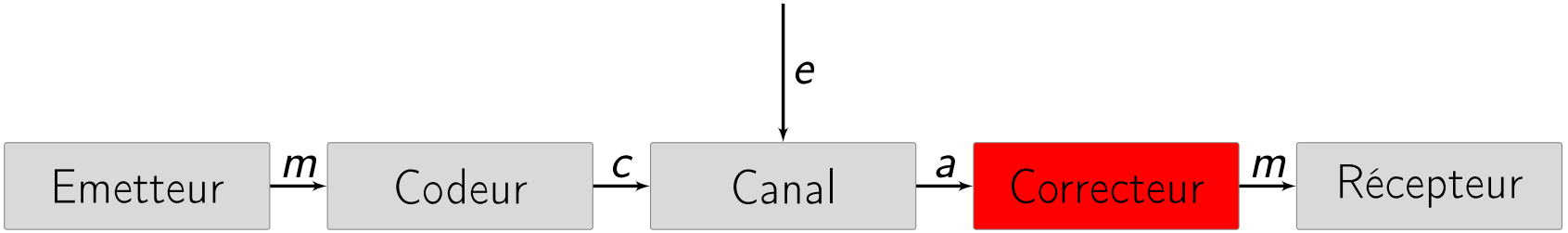


Figure – Représentation de la transmission du message

Décodage

Pour le décodage du code Reed Muller (2,4) on utilise la méthode du **vote majoritaire**.

Capacité de correction de $RM(r, m)$: $2^{m-r-1} - 1$.

1 Codes Correcteurs d'erreurs
- Principe
- Décodage d'un code

2 Code de Reed-Muller
- Encodage
- Décodage

3 Application
- Protocole de l'expérience
- Résultats

Encodage d'une image

$$\begin{pmatrix} 100 & 105 & 200 \\ 45 & 204 & 0 \\ 94 & 0 & 204 \end{pmatrix}$$

$\downarrow$ encodage sur 11 bits : fonction **decoupage**

$$\begin{pmatrix} [0,0,0,0,1,1,0,0,1,0,0] & [0,0,0,0,1,1,0,1,0,0,1] & [0,0,0,1,1,0,0,1,0,0,0] \\ [0,0,0,0,0,1,0,1,1,0,1] & [0,0,0,1,1,0,0,1,1,0,0] & [0,0,0,0,0,0,0,0,0,0,0] \\ [0,0,0,0,1,0,1,1,1,1,0] & [0,0,0,0,0,0,0,0,0,0,0] & [0,0,0,1,1,0,0,1,1,0,0] \end{pmatrix}$$

$\downarrow$ encodage grâce au code $RM(2,4)$: fonction **encodage**

$$\begin{pmatrix} [1,0,0,1,1,0,1,0,0,1,1,0,1,0,1,0] & [0,1,1,1,1,0,0,0,0,0,1,0,0,0,1,0] & [1,1,0,0,1,1,0,0,0,1,1,0,0,1,1,0] \\ [0,0,0,1,0,0,1,0,0,1,0,0,1,0,0,0] & [0,0,0,0,1,1,0,0,1,0,1,0,0,1,1,0] & [0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0] \\ [1,0,1,0,1,1,0,0,1,1,0,0,1,0,1,0] & [0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0] & [0,0,0,0,1,1,0,0,1,0,1,0,0,1,1,0] \end{pmatrix}$$

Figure – Représentation de l'encodage d'une image

Decodage d'une image

$$\begin{pmatrix} [1,0,0,1,1,0,1,0,0,1,1,0,1,0,1,0] & [0,1,1,1,1,0,0,0,0,0,1,0,0,0,1,0] & [1,1,0,0,1,1,0,0,0,1,1,0,0,1,1,0] \\ [0,0,0,1,0,0,1,0,0,1,0,0,1,0,0,0] & [0,0,0,0,1,1,0,0,1,0,1,0,0,1,1,0] & [0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0] \\ [1,0,1,0,1,1,0,0,1,1,0,0,1,0,1,0] & [0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0] & [0,0,0,0,1,1,0,0,1,0,1,0,0,1,1,0] \end{pmatrix}$$

$\downarrow$ ajout des erreurs : fonction **creation_erreurs**

$$\begin{pmatrix} [1,0,0,1,1,0,1,0,0,0,1,0,1,0,1,0] & [0,1,1,1,1,0,0,0,0,0,1,0,0,0,1,0] & [1,1,0,0,1,1,0,0,0,1,1,0,0,1,1,0] \\ [0,0,0,1,0,0,1,0,0,1,0,0,1,0,0,0] & [0,0,0,0,1,1,0,0,1,0,1,0,0,1,1,0] & [0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0] \\ [1,0,1,1,1,1,0,0,1,0,0,0,1,0,1,0] & [1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0] & [0,0,0,0,1,1,0,0,1,0,1,0,0,1,1,0] \end{pmatrix}$$

$\downarrow$ décodage du code $RM(2,4)$: fonction **decodage**

$$\begin{pmatrix} 100 & 105 & 200 \\ 45 & 204 & 0 \\ 94 & 0 & 204 \end{pmatrix}$$

Figure – Représentation du décodage d'une image

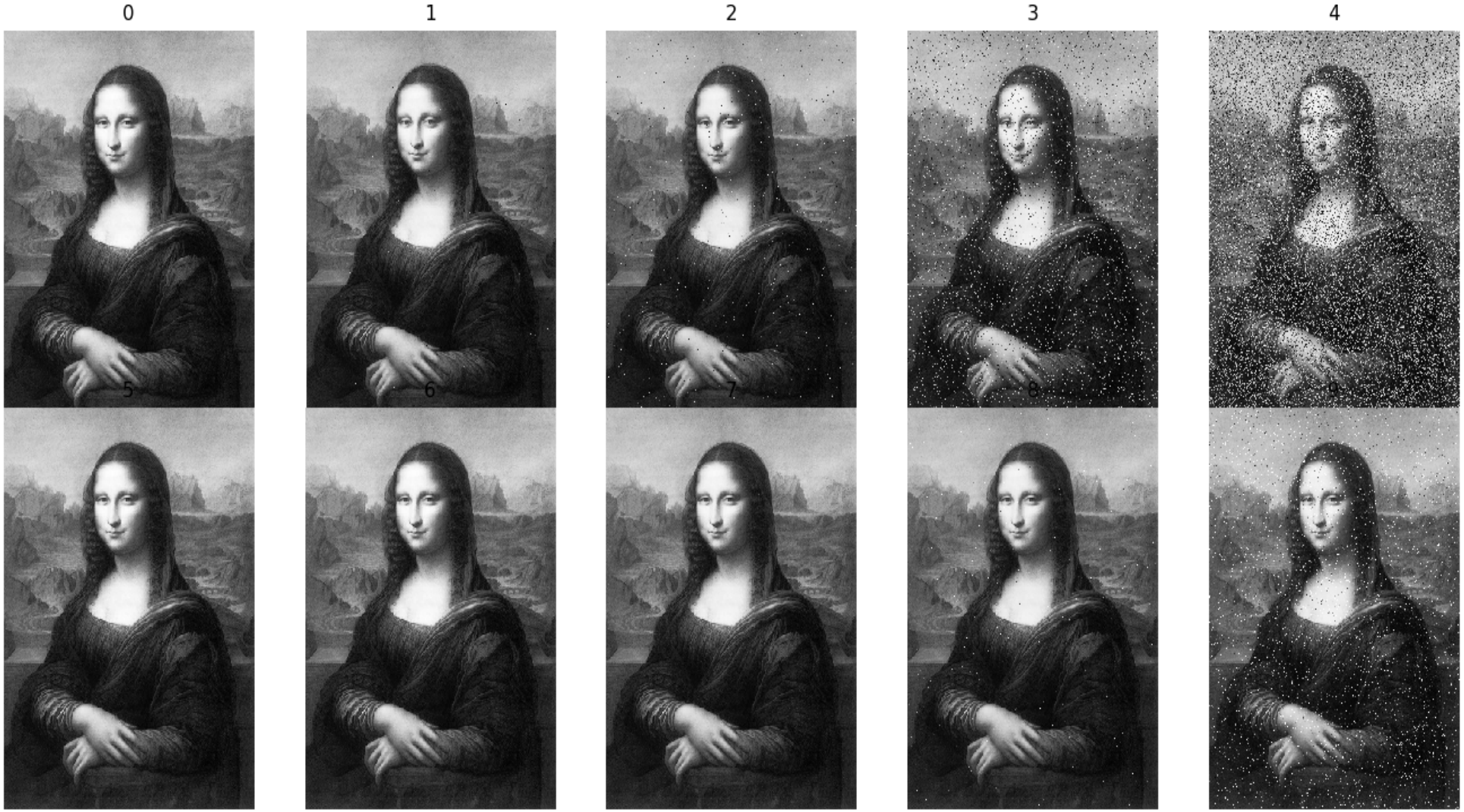
Image test

Figure – **La Joconde** de Leonard De Vinci

Quantité d'erreurs introduites

Nombre d'erreurs	100	1000	10000	25000	50000
Nombre de pixels de l'image	125000	125000	125000	125000	125000

Brouillage avec une répartition aléatoire des erreurs



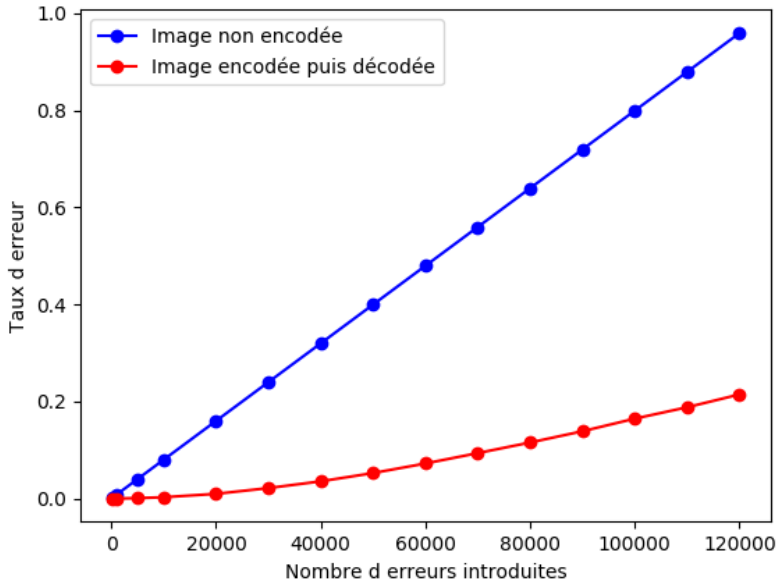


Figure – Taux d'erreur en fonction du nombre d'erreurs introduites

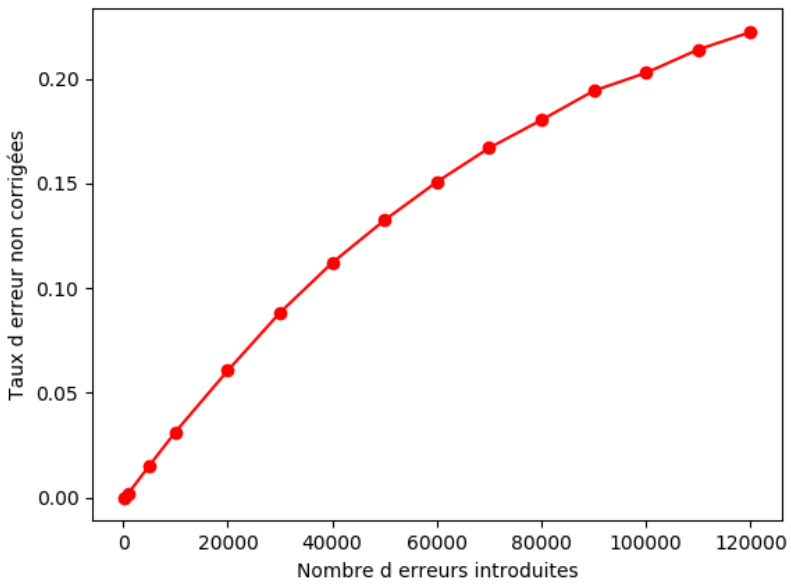
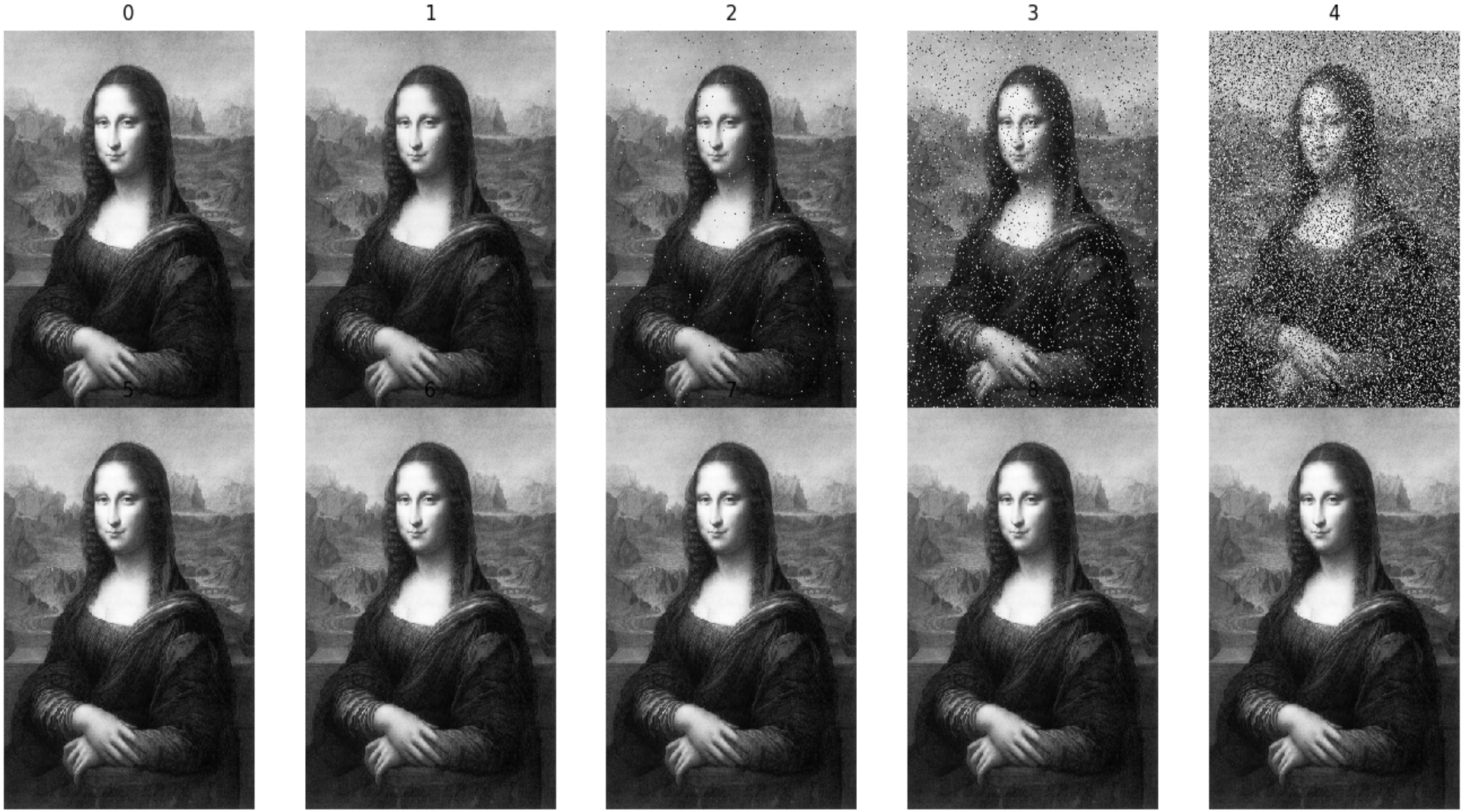


Figure – Taux d'erreur non corrigées en fonction du nombre d'erreurs introduites

Brouillage avec une répartition optimisée des erreurs



FIN

Code TIPE (2,4).py

```
## MODULES
import numpy as np
from copy import deepcopy
from random import randint
import matplotlib.pyplot as plt


## Création de la matrice génératrice pour encoder

# Fonction qui prend en entrée une liste représentant un entier en base 2 et qui le convertit en base 10.
def base10(L):
    x = 0
    for i in range(len(L)):
        x += L[i] * 2**(len(L)-i-1)
    return x


#Fonction réciproque, qui prend en entrée un entier et renvoie une liste représentant sa décomposition binaire.
def conv_base_2(n,b):
    L=[]
    while b>0:
        L.append(b%2)
        b=b//2
    for X in range(n-len(L)):
        L.append(0)
    L.reverse()
    return L


# Fonction qui énumère les éléments de F2_m qui correspondent à l'écriture binaire des entiers de 0 à (2**m)-1.
def F2_m(m):
    L=[]
    for i in range(2**m):
        L.append(conv_base_2(m,i))
    return L


# Fonction qui créée les m vecteurs à partir desquels on construit la matrice génératrice : on note ces vecteurs
```

```
v_1 , v_2 , ... , v_m et on ajoute au début le vecteur v0
qui ne contient que des 1.
def hyperplans(m):
    F2 = F2_m(m)
    liste_hyperplans=[[1 for i in range(2**m)]]
    for j in range(m):
        H=[]
        for i in range(2**m):
            if F2[i][j]:
                H.append(0)
            else:
                H.append(1)
        liste_hyperplans.append(H)
    return liste_hyperplans


# Fonction qui fait la multiplication terme à terme
de 2 vecteurs v_i et v_j qui sont contenus dans la liste
des m vecteurs.
def produit_exterieur(v,i,j):
    L=[]
    for k in range(len(v[i])):
        L.append(v[i][k]*v[j][k])
    return L


# Fonction qui créée la matrice génératrice en
mettant les vecteurs v0,...,v_m pour les m premières
lignes puis fait les produits extérieurs d'au plus r des
vi pour compléter la matrice, on s'arrête ici à l'ordre 2.
def matrice_generatrice_RM(r,m):
    v = hyperplans(m)
    if r==1:
        return np.array(v)
    elif r==2:
        matrice_g = v.copy()
        for i in range(1,m):
            for j in range(1,m+1-i):
matrice_g.append(produit_exterieur(v,i,i+j))
        return np.array(matrice_g)
    else:
        return "erreur"

RM = matrice_generatrice_RM(2,4)
```

```
## Fonctions pour faire des opérations sur les vecteurs

# Fonction qui fait le produit scalaire de 2 vecteurs modulo 2.
def prod_scal_mod2(v1,v2):
    n = len(v1)
    s = 0
    for i in range(n):
        s = s + v1[i]*v2[i]
    return s%2


# Fonction qui multiplie un vecteur ( une matrice à 1 ligne ) et une matrice.
def produit_vecteur_matrice(v,N):
    m,p = N.shape
    P = np.array([0 for i in range(p)])
    for j in range(p):
        P[j] = prod_scal_mod2(v,N[:,j])
    return P


# Fonction qui prend des listes/vecteurs et les additionne modulo 2.
def addition_mod2(M,N):
    taille = len(M)
    S = np.array([0 for i in range(taille)])
    for i in range(taille):
        S[i] = (M[i]+N[i])%2
    return S


## DECODAGE

# Fonction qui inverse les 0 et les 1 du vecteur.
def complementaire(x):
    c = x.copy()
    for i in  range(len(c)):
        if c[i]:
            c[i]=0
        else:
            c[i]=1
    return c


```

```
# Fonction qui détermine les vecteurs caractéristiques d'un monôme de degré 2 dont la ligne est la ligne i.
def vecteur_caracteristique_degre_2(r,m,i):
    indices = []
    vecteur = []
    cara = []

    #on détermine les monômes de degré 1 qui n'ont pas été utilisés pour créer RM[i]
    if i>=8 and i<=10:
        indices.append(1)
        indices.append(12-i)
    elif i>=6 and i<=7:
        indices.append(2)
        indices.append(10-i)
    else:
        indices.append(3)
        indices.append(4)

    #ajout du complémentaire
    for k in range(len(indices)):
        cara.append(RM[indices[k]])
        cara.append(complementaire(RM[indices[k]]))

    couples = [(0,2),(0,3),(1,2),(1,3)]
    for X in couples :
        a,b = X
        vecteur.append(cara[a]*cara[b])
    return vecteur


def couples_degre_1():
    P=[]
    for a in range(0,2):
        for b in range(2,4):
            for c in range(4,6):
                P.append((a,b,c))
    return P


# Fonction qui détermine les vecteurs caractéristiques d'un monôme de degré 1 dont la ligne est la ligne i.
def vecteur_caracteristique_degre_1(r,m,i):
    L=[]
```

```
    v = hyperplans(m)
    caracteristique = []

    #sépare les vecteurs v_j avec i!=j
    for j in range(1,len(v)):
        if j!=i:
            L.append(v[j])
            L.append(complementaire(v[j]))

    L = np.array(L)
    couples = couples_degre_1()
    for X in couples :
        a,b,c = X
        caracteristique.append(L[a]*L[b]*L[c])
    return caracteristique


def couples_degre_0():
    L = []
    for i in range(2):
        for j in range(2,4):
            for h in range(4,6):
                L.append((i,j,h))
    return L


# Fonction qui détermine les vecteurs caractéristiques du monôme de degré 0 correspondant à la première ligne de la matrice.
def vecteur_caracteristique_degre_0(r,m):
    v = np.eye(2**m)
    caracteristique = []
    for X in range(2**m):
        caracteristique.append(v[0])
    return caracteristique


# Fonction qui créée la liste complète des vecteurs caractéristiques de chaque monôme de la matrice, elle est appelée une et une seule fois d'où l'utilisation d'un tuple pour assurer l'intégrité des données pendant l'exécution du programme.
def creation_vecteur_caracteristique(r,m):
    vecteur_caracteristique = [vecteur_caracteristique_degre_0(r,m)]
    for i in range(1,m+1):
```

```
vecteur_caracteristique.append(vecteur_caracteristique_degre_1(r,m,i))
    for j in range(m+1,11):
vecteur_caracteristique.append(vecteur_caracteristique_degre_2(r,m,j))
    return vecteur_caracteristique

vecteur_caracteristique = tuple(creation_vecteur_caracteristique(2,4))


# Fonction qui traite la ligne i
def vote_majoritaire(r,m,i,message):
    liste_v_cara = vecteur_caracteristique[i]
    extraction = 0
    for X in liste_v_cara:
        extraction += prod_scal_mod2(message,X)
    if extraction < len(liste_v_cara)//2:
        return 0
    else:
        return 1

RM = matrice_generatrice_RM(2,4)
# Fonction qui décode, avec la méthode du vote majoritaire, un vecteur préalablement encodé par la matrice génératrice du code RM(r,m), ici il n'est efficace que pour le code RM(2,4).
def decodage_vecteur(r,m,message):
    message_decode = np.array([None for i in range(len(RM))])

    # Calcul des monômes de degré 2
    for i in range(len(RM)-1,m,-1):
        message_decode[i] = vote_majoritaire(r,m,i,message)

    # Les monômes de degre 2 ont été traités : on modifie le message
    S1 = produit_vecteur_matrice(message_decode[m+1:],RM[m+1:])
    E1 = addition_mod2(S1,message)

    # On traite ensuite les monômes de degré 1
    for j in range(m,0,-1):
```

```
        message_decode[j] = vote_majoritaire(r,m,j,E1)

    # Les monômes de degré 1 ont été traités : on modifie le message
    S2 = produit_vecteur_matrice(message_decode[1:m+1],RM[1:m+1])
    E2 = addition_mod2(S2,E1)

    # Calcul des monômes de degre 0
    message_decode[0] = vote_majoritaire(r,m,0,E2)
    return message_decode



## Conversion de l'image


def gris(p):
    return np.uint8(round(np.mean(p)))

# Fonction qui convertit une image en niveau de gris.
def conversion(a):
    ligne, colonne,autre = np.shape(a)
    g = np.zeros((ligne,colonne), dtype=np.uint8 )
    for i in range(ligne):
        for j in range(colonne):
            g[i,j] = gris(a[i,j])
    return g


## Encodage du message

# Fonction qui transforme un vecteur de vecteur en une liste d'entiers pour faciliter l'encodage de chaque pixel.
def aplatir(image):
    L=[]
    for X in image:
        for Y in  X:
            L.append(Y)
    return L


# Fonction qui convertit une image, i.e , une matrice de pixels en une liste de liste de bits.
```

```
273| def decoupage(image):
274|     # on sépare les pixels et on en fait une liste
275|     L=[]
276|     for X in image:
277|         for Y in  X:
278|             # chaque pixel est codé sur 11 bits
279|             L.append(conv_base_2(11,Y))
280|     return L
281|
282|
283| # Fonction qui encode une image en traitant chaque
liste de 11 bits.
284| def encodage(image):
285|     liste_pixels = decoupage(image)
286|     RM = matrice_generatrice_RM(2,4)
287|     for i in range(len(liste_pixels)):
288|         liste_pixels[i] =
produit_vecteur_matrice(liste_pixels[i],RM)
289|     return liste_pixels
290|
291|
292| # Fonction réciproque de decoupage, elle prend en
entrée une liste de vecteurs qui correspondent aux listes
de 11 bits précédemment citées, qui ont été décodées.
293| def recollage(image,hauteur,largeur):
294|     copie,L = aplatir(image), []
295|
296|     # reconversion de chaque liste de 11 bits en
l'entier correspondant
297|     for k in range(0,len(copie),11):
298|         L.append(base10(copie[k:k+11]))
299|
300|     matrice = np.zeros((hauteur,largeur),
dtype=np.uint8 )
301|     indice_ligne = 0
302|
303|     # restauration de la matrice de départ qui
représente l'image.
304|     for x in range(0,len(L)):
305|         if x!=0 and not(x%largeur):
306|             indice_ligne +=1
307|         matrice[indice_ligne][x%largeur] = L[x]
308|     return matrice
309|
310|
311| # Fonction réciproque de encodage, elle prend en
```

```
entrée une liste de vecteurs de 11 bits.
312| def decodage(image,hauteur,largeur):
313|     image_decode  = []
314|
315|     # chaque vecteur est décodé grâce au code RM(2,4)
316|     for X in image:
317|         if len(X)==16:
318|
image_decode.append(decodage_vecteur(2,4,X))
319|         else:
320|             image_decode.append(X)
321|     # l'image est reformée
322|     image_decode =
recollage(image_decode,hauteur,largeur)
323|     return np.array(image_decode)
324|
325|
326|
327| ## Génération des erreurs
328|
329|
330| # Fonction qui introduit aléatoirement des erreurs
dans une image et garde en mémoire la localisation de ces
erreurs.
331| def creation_erreurs(image,nb_erreurs):
332|     lignes,colonnes = np.shape(image)
333|     localisation_erreurs = []
334|     image_alteree = deepcopy(image)
335|
336|     for i in range(nb_erreurs):
337|         e_ligne,e_colonne,couleur = randint(0,
lignes-1),randint(0, colonnes-1),randint(0, 255)
338|         localisation_erreurs.append( e_ligne*colonnes
+ e_colonne )
339|         image_alteree[e_ligne][e_colonne] = couleur
340|
341|     return image_alteree,localisation_erreurs
342|
343|
344| # Fonction qui introduit des erreurs selon une
répartition déterminée au préalable.
345| def
creation_erreurs_localisee(image,localisation_erreurs):
346|     image_alteree = deepcopy(image)
347|     for X in localisation_erreurs:
348|         a = randint(0, 16)
```

```
349|         try:
350|             if image_alteree[X][a]:
351|                 image_alteree[X][a] = 0
352|             else:
353|                 image_alteree[X][a] = 1
354|         except:
355|             return (X,a)
356|     return image_alteree
357|
358|
359| # Fonction qui introduit au maximum une erreur par
blocs de 16 bits.
360| def creation_erreurs_optimisee(image,erreurs):
361|     lignes,colonnes = np.shape(image)
362|     localisation_erreurs = []
363|     image_alteree = np.copy(image)
364|     nb_erreurs = min(lignes*colonnes,erreurs)
365|
366|     for i in range(nb_erreurs):
367|
368|         e_ligne,e_colonne,couleur = randint(0,
lignes-1),randint(0, colonnes-1),randint(0, 255)
369|
370|         while ((e_ligne*colonnes + e_colonne) in
localisation_erreurs):
371|             e_ligne,e_colonne,couleur = randint(0,
lignes-1),randint(0, colonnes-1),randint(0, 255)
372|
373|         localisation_erreurs.append( e_ligne*colonnes
+ e_colonne )
374|         image_alteree[e_ligne][e_colonne] = couleur
375|
376|     return image_alteree,localisation_erreurs
377|
378|
379|
380|
381|
382|
383|
384|
385|
386|
387|
```